Lecture 7 — Prefix Sums, 2D Prefix Sums & Difference Arrays

> Year Curriculum · Prerequisites: Weeks 2–6 (STL maps, complexity, two pointers) · Estimated time: 2 hours lecture + 4 hours practice

1. Overview

Prefix sums are the simplest form of **precomputation**: spend O(n) once so that every later question can be answered in O(1). They are among the most frequently used tricks in competitive programming, and they combine with almost everything else in this course: binary search (Week 5), dynamic programming (Weeks 14–15), and, through the difference array, the idea behind range updates in Fenwick and segment trees (Weeks 23–25).

This lecture covers:

1. 1D prefix sums and range-sum queries.
2. Generalizing to XOR, counts, and other invertible operations.
3. Counting subarrays with a given sum or remainder by matching **pairs of prefixes**, which works where two pointers fail.
4. 2D prefix sums and rectangle queries via inclusion–exclusion.
5. **Difference arrays**, the inverse of prefix sums, for applying many range updates at once, in 1D and 2D and stacked.

2. One-dimensional prefix sums

2.1 Definition

For a 1-indexed array a[1..n], define

p[0] = 0, p[i] = p[i−1] + a[i], so that p[i] = a[1] + … + a[i].

Then for 1 ≤ l ≤ r ≤ n:

sum(a[l..r]) = p[r] − p[l−1],

because p[r] contains a[1..r], and subtracting p[l−1] removes a[1..l−1].

Example:

i	0	1	2	3	4	5
a[i]		3	1	4	1	5
p[i]	0	3	4	8	9	14

sum(2..4) = p[4] − p[1] = 9 − 3 = 6 = 1 + 4 + 1.

Why p[0] = 0 matters. With the sentinel, the formula also works for l = 1, giving p[r] − p[0] = p[r], with no special case. With a 0-indexed array, the usual convention is p of size n + 1 with p[i+1] = p[i] + a[i], and sum(a[l..r]) = p[r+1] − p[l].

2.2 Complexity

Building takes O(n) and each query O(1). For n, q = 2·10^5 (CSES 1646) this is about 4·10^5 operations, compared with 4·10^10 for naive summation. The sum can reach 2·10^5 · 10^9 = 2·10^14, so prefix arrays must be `long long`.

2.3 Other operations

The trick works for any associative operation with an **inverse**:

- **XOR**: x[r] ^ x[l−1] gives the XOR of a[l..r], because XOR is its own inverse (CSES 1650).
- **Counts**: to count positions in [l, r] that satisfy a property, build a 0/1 indicator array and prefix-sum it. In **CF 313B (Ilya and Queries)**, the property is s[i] = s[i+1].
- **Products modulo a prime**: use modular inverses, covered in Week 11. Beware of zeros.
- **Min/max**: there is **no inverse**, so prefix minima do not answer arbitrary range-minimum queries. Use a sparse table (Week 24) or a segment tree (Week 23). Prefix minima still help in specific scans, such as the maximum subarray sum in Section 3.3.

Several prefix arrays can be built side by side. **CF 433B (Kuriyama Mirai's Stones)** needs range sums of both the original array and its sorted copy, so it uses two prefix arrays.

3. Counting subarrays with prefix sums

A **subarray** corresponds to a **pair of prefixes**: a[l..r] ↔ (p[l−1], p[r]). Any condition on the subarray sum becomes a condition on two prefix values. Counting subarrays then means counting pairs, which a hash map handles in one pass.

3.1 Subarrays with sum exactly x — CSES 1661 Subarray Sums II

Values may be **negative**, so the two-pointer method of Week 6 does not apply. Instead:

sum(l..r) = x ⇔ p[l−1] = p[r] − x.

Scan r from 1 to n. Keep a map `seen` holding how many times each prefix value occurred among p[0..r−1], and add `seen[p[r] − x]` to the answer. Initialize `seen[0] = 1` to account for p[0], which covers subarrays that start at index 1.

Worked example: a = [2, −1, 3, −2, 1], x = 2. Prefixes: 0, 2, 1, 4, 2, 3.

r	p[r]	need p[r]−x	seen before	add	running ans
1	2	0	{0:1}	1	1 ([2])
2	1	−1	{0:1, 2:1}	0	1
3	4	2	{0:1, 2:1, 1:1}	1	2 ([−1, 3])
4	2	0	{…, 4:1}	1	3 ([2, −1, 3, −2])
5	3	1	{0:1, 2:2, 1:1, 4:1}	1	4 ([3, −2, 1])

Complexity: O(n log n) with `map`, or expected O(n) with `unordered_map` and a safe hash (Week 2). The answer can be about $n^2/2$, so store it in `long long`.

3.2 Subarrays with sum divisible by n — CSES 1662 Subarray Divisibility

sum(l..r) ≡ 0 (mod n) ⇔ p[r] ≡ p[l−1] (mod n).

Count pairs of prefixes with equal remainders. Since remainders lie in [0, n), a plain array of counts suffices. **Trap:** in C++, `(-7) % 3 == -1`. Normalize with `((v % n) + n) % n` whenever values can be negative.

The same idea solves **USACO 2016 January Silver, "Subsequences Summing to Sevens"**: find the longest subarray with sum divisible by 7 by recording the **first** index at which each remainder appears.

3.3 Maximum subarray sum — CSES 1643

For a fixed r, the best subarray ending at r is p[r] − min(p[0..r−1]). Keep a running minimum of the prefixes seen so far. This is Kadane's algorithm seen through prefix sums. It takes O(n) and correctly handles all-negative arrays, because the subarray must be non-empty and we update `best` before updating the minimum.

4. Two-dimensional prefix sums

For a grid a[1..n][1..m], let P[i][j] be the sum of the rectangle from (1, 1) to (i, j). It satisfies

P[i][j] = a[i][j] + P[i−1][j] + P[i][j−1] − P[i−1][j−1].

The two rectangles P[i−1][j] and P[i][j−1] overlap in P[i−1][j−1], which was added twice, so it is subtracted once. With a zero row and column at index 0, the recurrence needs no boundary checks.

Rectangle query. The sum over rows r1..r2 and columns c1..c2 is

P[r2][c2] − P[r1−1][c2] − P[r2][c1−1] + P[r1−1][c1−1].

This is **inclusion–exclusion**. Take the big rectangle, remove the strip above the query and the strip to its left, then add back the top-left corner, which was removed twice.

Example (3×3 grid):

```
a:  1 2 3      P:  1  3  6
    4 5 6          5 12 21
    7 8 9         12 27 45
```

Sum of rows 2..3, columns 2..3 = P[3][3] − P[1][3] − P[3][1] + P[1][1] = 45 − 6 − 12 + 1 = 28 = 5 + 6 + 8 + 9.

Building takes O(nm) and each query O(1). **CSES 1652 Forest Queries** counts trees in rectangles of a 1000 × 1000 grid with 2·10^5 queries. Note that it gives coordinates as (y, x).

A common extension: find the maximum-sum k × k square, or count sub-rectangles with a property, by enumerating corners and using O(1) rectangle sums.

5. Difference arrays

5.1 The idea

Prefix sums answer many **queries** on a fixed array. Difference arrays handle the opposite situation: many **updates** of the form "add v to every element of a[l..r]", followed by reading the final array once.

Define d[i] = a[i] − a[i−1], with a[0] = 0. Then a is the prefix sum of d. Adding v to a[l..r] changes only two differences:

- d[l] increases by v, since a[l] went up and a[l−1] did not;
- d[r+1] decreases by v, since a[r] went up and a[r+1] did not.

So each range update costs **O(1)**: `d[l] += v; d[r + 1] -= v;`. One prefix-sum pass at the end rebuilds a in O(n). Allocate d with size n + 2 so that index r + 1 is always valid.

Trace, n = 5, starting from zeros:

step	d[1]	d[2]	d[3]	d[4]	d[5]	d[6]
+2 on [2,4]	0	2	0	0	−2	0
+3 on [1,2]	3	2	−3	0	−2	0
prefix → a	3	5	2	2	0	

5.2 Coverage counting — CF 816B Karen and Coffee

Each of n recipes recommends a temperature interval [l_i, r_i], and a temperature is *admissible* if at least k recipes recommend it. Each query asks how many admissible integer temperatures lie in [a, b]. Temperatures go up to 200000.

1. Difference array: for each recipe, `d[l]++`, `d[r+1]--`. Its prefix sum gives the coverage of every temperature.
2. Indicator: ok[t] = (coverage[t] ≥ k).
3. Prefix sums of ok. Each query is then a single subtraction.

Total O(n + q + MAXT). The pattern "difference array → derived array → prefix sums" appears often.

AtCoder ABC014 C (AtColor) is the same coverage count: find the maximum number of intervals covering a point.

5.3 Stacked difference arrays — CF 295A Greg and Array

There are m operations "add d_i to a[l_i..r_i]" and k queries "apply operations x through y". First run a difference array over operation indices to find how many times `times[i]` each operation is applied. Then run a difference array over array positions, adding `times[i] · d_i` on [l_i, r_i]. Finally take a prefix sum. Total O(n + m + k). Values reach about 10^5 · 10^5 · 10^5 = 10^15, so use `long long`.

5.4 Two-dimensional difference arrays

To add v to the rectangle (r1, c1)–(r2, c2):

```
D[r1][c1]       += v;
D[r1][c2 + 1]   -= v;
D[r2 + 1][c1]   -= v;
D[r2 + 1][c2 + 1] += v;
```

Taking the 2D prefix sum of D produces the final grid. This is the same inclusion–exclusion pattern, applied in reverse. The classic use is "after placing many rectangles, how many cells are covered by exactly k of them?", as in **USACO 2019 February Silver, "Painting the Barn"**.

5.5 Prefix sums and difference arrays are inverses

Prefix sum (∑) and difference (Δ) undo each other: ∑Δa = a and Δ∑a = a. A **point update** on Δa is a **range update** on a, and reading a single value a[i] is a **prefix-sum query** on Δa. This duality returns in Week 24, where a Fenwick tree over a difference array supports range updates with point queries.

5.6 Limits

Prefix sums require the array to be **static** between queries, and difference arrays require all updates to come **before** the reads. If updates and queries interleave online, each operation would cost O(n). That is the motivation for Fenwick trees and segment trees (Weeks 23–25), which do both in O(log n).

6. Reference implementations

6.1 Static Range Sum Queries (CSES 1646)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> p(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        p[i] = p[i - 1] + x;
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << p[r] - p[l - 1] << '\n';
    }
    return 0;
}
```

6.2 Subarray Sums II (CSES 1661)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long x;
    cin >> n >> x;
    map<long long, int> seen;
    seen[0] = 1;                        // the empty prefix p[0] = 0
    long long p = 0, ans = 0;
    for (int i = 0; i < n; i++) {
        long long a;
        cin >> a;
        p += a;
        auto it = seen.find(p - x);
        if (it != seen.end()) ans += it->second;
        seen[p]++;
    }
    cout << ans << '\n';
    return 0;
}
```

Sample: `5 7` / `2 -1 3 5 -2` → `2`.

6.3 Subarray Divisibility (CSES 1662)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> cnt(n, 0);
    cnt[0] = 1;
    long long p = 0, ans = 0;
    for (int i = 0; i < n; i++) {
        long long a;
        cin >> a;
        p = ((p + a) % n + n) % n;      // normalize negative remainders
        ans += cnt[p];
        cnt[p]++;
    }
    cout << ans << '\n';
    return 0;
}
```

Sample: `5` / `3 1 2 7 4` → `1`.

6.4 Forest Queries (CSES 1652)

```cpp
#include <bits/stdc++.h>
using namespace std;

int P[1001][1001];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; i++) {
        string row;
        cin >> row;
        for (int j = 1; j <= n; j++)
            P[i][j] = (row[j - 1] == '*') + P[i - 1][j] + P[i][j - 1] - P[i - 1][j - 1];
    }
    while (q--) {
        int y1, x1, y2, x2;
        cin >> y1 >> x1 >> y2 >> x2;
        cout << P[y2][x2] - P[y1 - 1][x2] - P[y2][x1 - 1] + P[y1 - 1][x1 - 1] << '\n';
    }
    return 0;
}
```

6.5 Karen and Coffee (CF 816B): difference array + prefix sums

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    const int MAXT = 200000;
    int n, k, q;
    cin >> n >> k >> q;
    vector<int> d(MAXT + 2, 0);
    for (int i = 0; i < n; i++) {
        int l, r;
        cin >> l >> r;
        d[l]++;
        d[r + 1]--;
    }
    vector<int> good(MAXT + 1, 0);     // prefix count of admissible temperatures
    int cover = 0;
    for (int t = 1; t <= MAXT; t++) {
        cover += d[t];
        good[t] = good[t - 1] + (cover >= k);
    }
    while (q--) {
        int a, b;
```

```
        cin >> a >> b;
        cout << good[b] - good[a - 1] << '\n';
    }
    return 0;
}
```

Sample: `3 2 4` / `91 94` / `92 97` / `97 99` / queries `92 94`, `93 97`, `95 96`, `90 100` → `3 3 0 4`.

7. Common mistakes

1. **Missing the sentinel p[0] = 0**, or `seen[0] = 1` in counting problems. This loses every subarray that starts at the first element.
2. **Overflow.** Prefix sums of 10^5 values of 10^9 exceed `int`. The 2D Forest grid fits in `int` (at most 10^6), but sums of values usually do not.
3. **Negative remainders.** C++ `%` keeps the sign of the dividend. Normalize.
4. **Off-by-one between 0- and 1-indexing.** Pick one convention per problem. 1-indexed with sentinels is the least error-prone.
5. **Size n + 1 for the difference array**, when you write to index r + 1 = n + 1. Allocate n + 2.
6. **2D coordinate order.** Statements may give (x, y) or (row, column). CSES 1652 gives y first.
7. **Using prefix sums for min/max range queries.** There is no inverse, so it does not work.
8. **Rebuilding prefix sums after every update.** That is O(n) per update. If updates interleave with queries, you need a Fenwick tree.
9. **`int` for counts of subarrays.** Up to about 2·10^10.

8. Practice problems

CSES (Range Queries and Sorting/Searching sections)

- 1646 — Static Range Sum Queries — https://cses.fi/problemset/task/1646
- 1650 — Range Xor Queries — https://cses.fi/problemset/task/1650
- 1652 — Forest Queries — https://cses.fi/problemset/task/1652
- 1661 — Subarray Sums II — https://cses.fi/problemset/task/1661
- 1662 — Subarray Divisibility — https://cses.fi/problemset/task/1662
- 1643 — Maximum Subarray Sum — https://cses.fi/problemset/task/1643

Codeforces

- 313B — Ilya and Queries — https://codeforces.com/problemset/problem/313/B
- 433B — Kuriyama Mirai's Stones — https://codeforces.com/problemset/problem/433/B
- 816B — Karen and Coffee — https://codeforces.com/problemset/problem/816/B
- 295A — Greg and Array — https://codeforces.com/problemset/problem/295/A

AtCoder

- ABC014 C — AtColor — https://atcoder.jp/contests/abc014/tasks/abc014_3

USACO

- 2016 January Silver — Subsequences Summing to Sevens — http://www.usaco.org/index.php?page=viewproblem2&cpid=595
- 2015 December Silver — Breed Counting — http://www.usaco.org/index.php?page=viewproblem2&cpid=572
- 2019 February Silver — Painting the Barn (2D difference array) — http://www.usaco.org/index.php?page=viewproblem2&cpid=919

9. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Section 9.1 ("Static array queries"): https://cses.fi/book/book.pdf
- USACO Guide, Silver modules "Introduction to Prefix Sums" and "More on Prefix Sums" (2D, difference arrays, prefix minimum): https://usaco.guide/silver/prefix-sums
- Codeforces EDU course "Segment Tree", whose introduction explains why prefix sums stop working once updates arrive
- Looking ahead: Week 23 (segment trees), Week 24 (Fenwick trees and sparse tables), Week 25 (lazy propagation)